Assignment Report – Group 5 (Alessandro D'Annibale)

01RKZOV – Testing and Fault Tolerance

Introduction

The goal of the assignment was to develop a Path Margin Monitor Unit (or PMU in short) to measure path delays in IC designs.

In digital circuits, signals propagate between sequential elements, such as flip flops and registers, according to timing determined by clocks. In order for digital circuits to function properly, the functional signals must arrive at the next sequential element at least a certain amount of time before the relevant clock signal. The difference between when the functional signal arrives and the latest time when it could arrive is referred to as the "timing margin" or "path margin" or simply "margin". During the design of ICs, the timing of the circuit is simulated to ensure that there is sufficient margin along the different signal paths in the IC. However, the actual margins in a fabricated chip may differ from the simulations, and the margins may change as a function of temperature and voltage. Moreover, the margins may also change over time as the chip ages.

A Path Margin Monitor (or PMM in short) can monitor several functional paths of the IC and characterize their margins over the product lifecycle. A PMM can be used during Manufacturing test on ATE to help designers to get a more accurate understanding on the functioning of the chip. A PMM can be also used during Mission mode, as it measures the degradation of silicon due to aging and the corresponding shortening of margins. Measured path margins may also be used to improve libraries and timing models.

A PMM consists of 2 different components: the Path Margin Controller (or PMC in short) and the PMU. PMUs are distributed across the IC design (from hundreds to thousands of units) and connected in daisy chain loops to the PMC, which enables the path margin measurements and data collections.

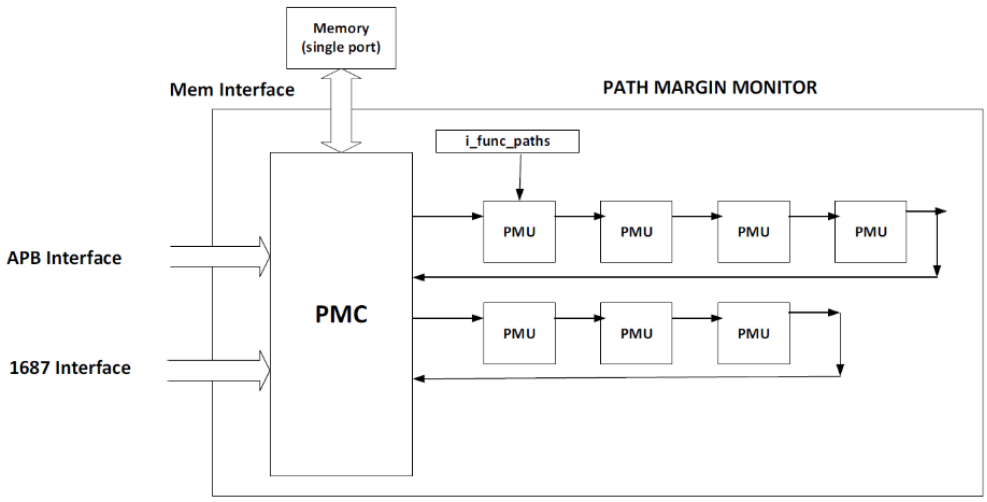

Figure 1. High-level Architecture of Path Margin Monitor

The PMU is connected to the functional paths at a point near the destination FF (for the assignment a maximum of 8 paths can be connected to the PMU). Only one path at the time can be selected and monitored using a path select MUX. Additional delay is introduced on the path by adding a series of delay elements such as buffers or back-to-back inverters, e.g. the "delay chain". Data is then latched into the "Shadow FF" without additional delay (before entering the delay chain) and into the "Capture FF" with the additional delay, which can be controlled by a delay select MUX.

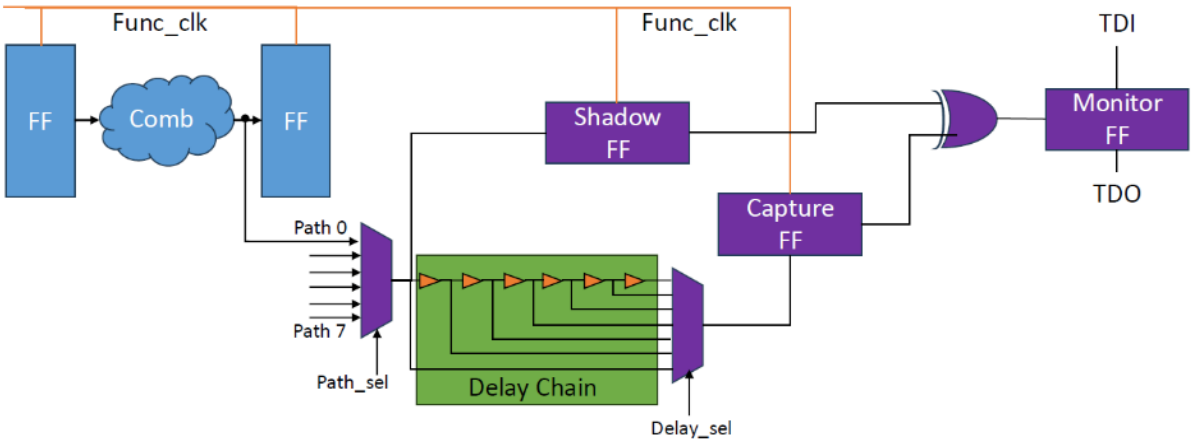

Figure 2. PMU basic working principle

Shadow FF and Capture FF are then compared using a XOR and if the result is 1 it means that the two FFs are not equal and the additional delay was enough to cause the data to arrive during setup window or after the clock edge so that it is not captured correctly into Capture FF. The result is then saved into the "Monitor FF", which will be read by the PMC together with the result of all the other PMUs in the IC. To find the margin of a path we need to start from a zero delay and then add delays incrementally until "Monitor FF" contains 1: the margin of the path will be between the selected delay and the previous one.

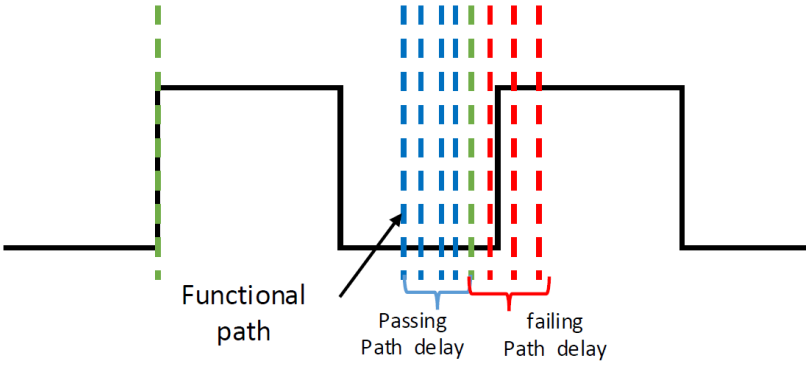

Figure 3. Timing Margins on Path under monitor

To maintain uniformity on the delay chain across different PMUs in the IC, it is crucial to implement the PMU as a macro cell that can be instantiated inside the IC. To develop the PMU macro cell, it was chosen to use IIC-OSIC-TOOLS (Integrated Infrastructure for Collaborative Open Source IC Tools), which is an all-in-one Docker/Podman container for open-source-based IC designs for analog and digital circuit flows. Of the three open-source process-development kits (PDKs) pre-installed in the Docker container, the SkyWater Open Source PDK (sky130) was chosen. All the used software inside the container will be listed in the following sections of the report.

How to setup the environment

1. Clone the PMU GitHub repository to your local machine or extract the provided ZIP archive.
2. Install Docker Desktop on your local machine (for Windows, WSL should be present too).
3. Start Docker.
4. Run the setup script ("setup_windows.bat" for Windows or "setup_linux.sh" for Linux) inside the PMU repository folder.
5. Wait until the Docker container opens.

The assignment was divided into three parts: the design of the delay chain, the design of the PMU, and the integration of the PMU into a simple IC (a 16-bit RCA). All the output files and plots can be generated with Bash scripts provided inside the folder of each step of the assignment.

The delay chain

For the first part of the assignment, the delay chain was created using Xschem, a schematic capture program which allows creation of hierarchical representation of circuits with a top-down approach. Inside the delay chain schematic, there are two inputs: in, the functional path to monitor, and delay_sel[2:0], the selection signal for the 8:1 MUX, which was implemented using the standard cells provided by the sky130 PDK, e.g. two 4:1 MUXes connected to one 2:1 MUX. One output is present, out, which drives the functional path with the selected delay.

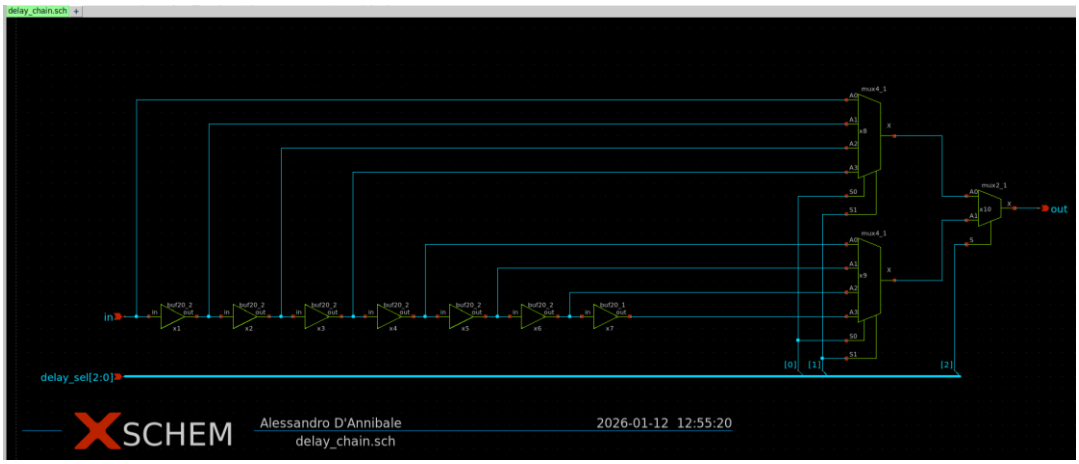

Figure 4. Delay Chain schematic (delay_chain.sch)

As for the delay elements used inside the delay chain, custom buffers (buf20_2 and buf20_1) were created using Xschem and instantiated inside the delay chain schematic. Each custom buffer contains a chain of 20 standard cell buffers, so that the delay can be observed in terms of nanoseconds.

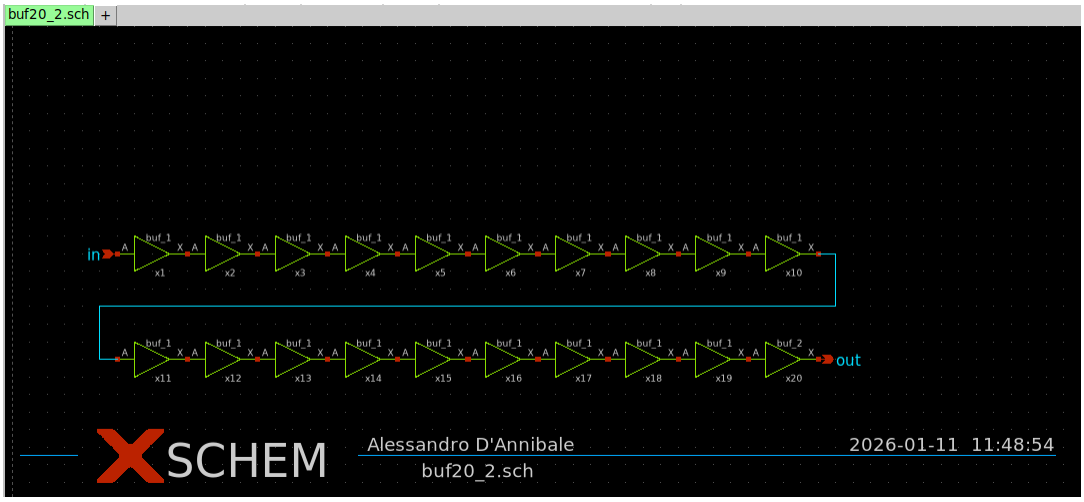

Figure 5. Custom Buffer schematic (buf20_2.sch)

After the creation of the delay chain schematic, Xschem was used to produce the SPICE and Verilog netlists. The first one (delay_chain.spice) is useful to make a SPICE simulation and to measure in a first approximation the obtained delays, whereas the second one (delay_chain.v) will be used for the synthesis of the PMU.

A SPICE testbench (delay_chain_tb.spice) was created to test the delay chain and it was run using ngspice, an open-source spice simulator for electric and electronic circuits. **Figure 6.** shows a plot generated by the testbench with a periodic input in red and the delayed output in blue. We can see how the delay grows constantly at every input period, as the delay selection signal changes every 20 ns, from 000 up until 111 is reached (8 different delays can be selected).

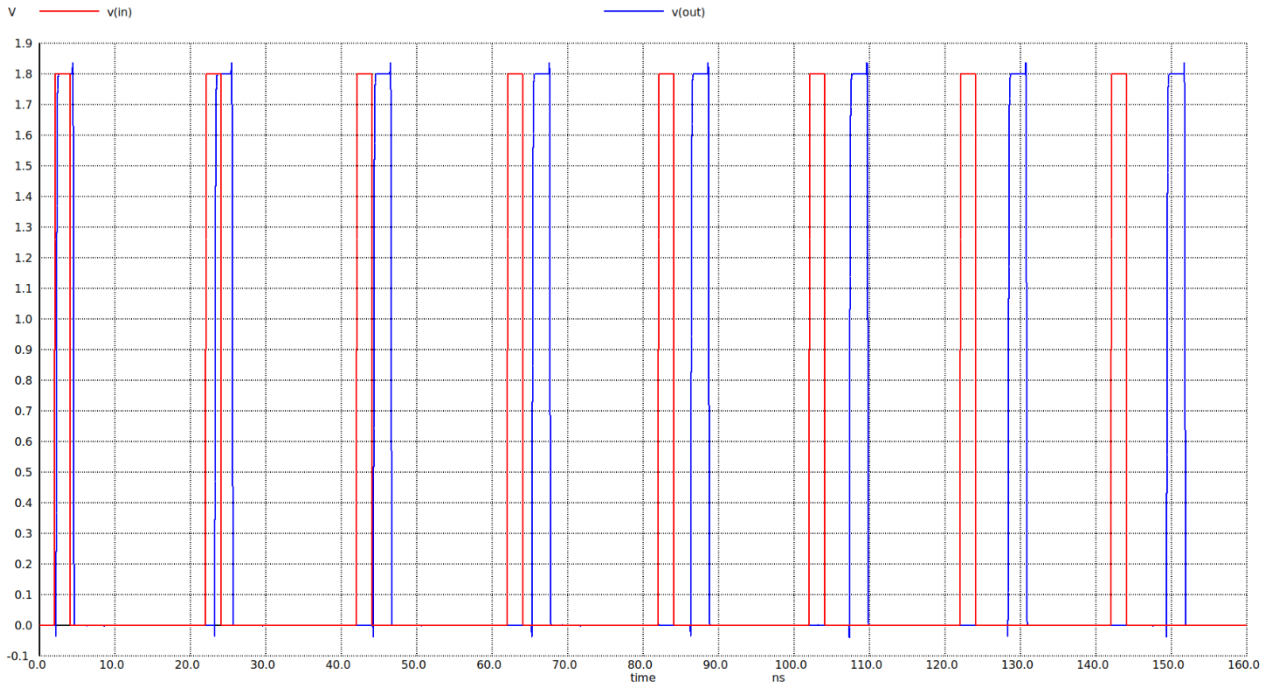

Figure 6. Delay Chain testbench plot

The propagation delays are not reported here, as they will change after inserting the delay chain into the PMU, but they can be read calling the “plot.sh” Bash script inside the “delay_chain/2-simulation” folder.

The PMU

For the second part of the assignment, the PMU was synthesized and its layout was produced with the LibreLane flow. LibreLane is an ASIC infrastructure library based on several open-source components (including OpenROAD, Yosys, Magic, Netgen, CVC, KLayout) and a number of custom scripts for design exploration and optimization. A reference flow, "Classic", performs all ASIC implementation steps from RTL all the way down to GDSII.

To configure the LibreLane flow, a “config.json” file was used, which specifies the design name, the Verilog files used (“delay_chain.v” and “pmu.v”, the latter containing the RTL description of the module with the instance of the delay chain), the clock name and period, the SDC file for constraints, the “pin_order.cfg” file for pin placement, the core utilization (70% to reduce the PMU area), and other flags to disable optimizations, as the added buffers should not be touched by the flow.

After the LibreLane flow finishes, the PMU layout is generated inside the “pmu.gds” file and can be visualized with KLayout, an open-source layout viewer and editor for IC and MEMS designs. A SPICE netlist of the PMU (“pmu_pex.spice”) can be obtained using Magic, a VLSI layout tool which can extract all the parasitics inside the circuit introduced with Place&Route. Then, another (more precise) SPICE simulation can be performed using ngspice and another testbench (pmu_pex_tb.spice). **Figure 8.** shows a plot generated by the testbench with an input in red with period 40ns driving path0 of the PMU, the input of Capture FF in blue, the output of Shadow FF and Capture FF in orange and green,

respectively, and the output of Monitor FF in pink. The delay selection signal changes every 40ns. When delay_sel[2:0] = 101, the mismatch between Shadow FF and Capture FF happens and the condition is made available by Monitor FF to the outside at the next clock cycle (the clock has a period of 10ns).

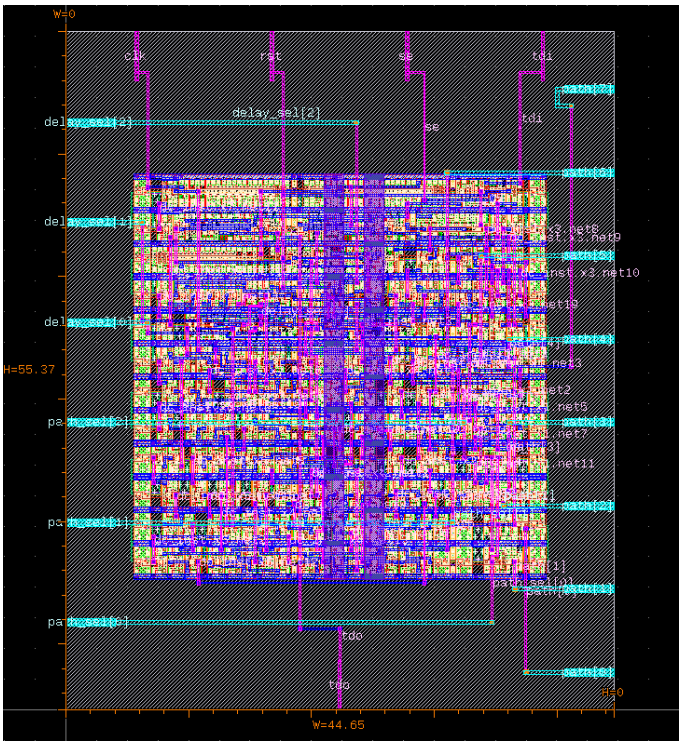

Figure 7. PMU Layout (H=55.37 µm, W=44.65 µm)

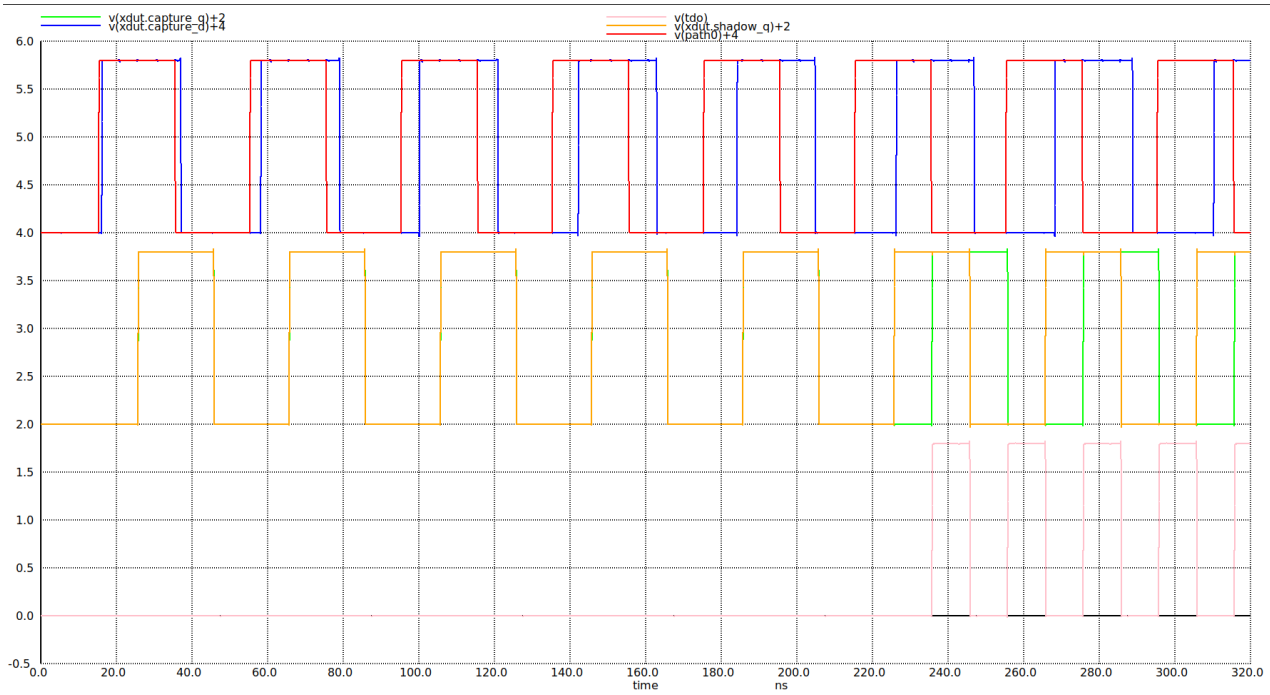

Figure 8. PMU testbench plot

The propagation delays are reported in **Figure 9.** and were obtained using the "plot.sh" Bash script inside the "pmu/3-simulation" folder. A slightly difference between the delay of a rising input signal and the corresponding falling signal is present and should be considered when approximating the margin of the monitored functional path.

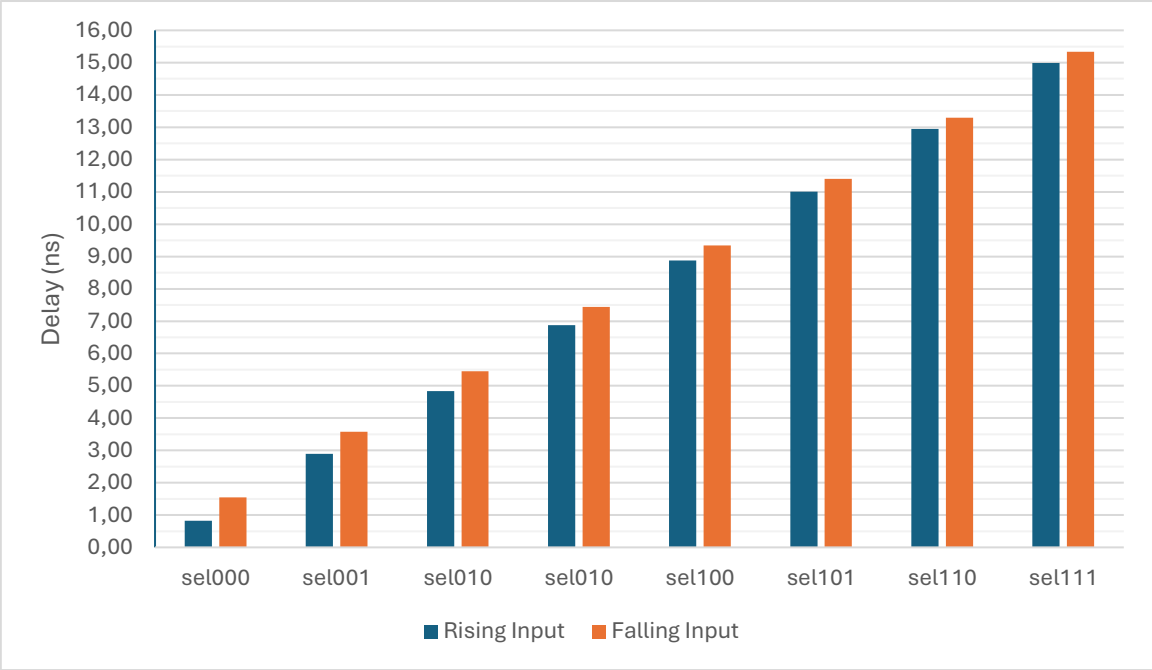

Figure 9. PMU Propagation Delays (in ns)

Integrating the PMU inside an RCA

For the third and final part of the assignment, the PMU macro cell was instantiated inside a 16-bit Ripple Carry Adder (RCA). Although being simple, the RCA is good for our purposes, because it has a long critical path, which goes from the LSB of the two input operands to the carry-out output. For this reason, the PMU was connected to the end of the functional critical path, before the carry-out destination FF.

The same flow used for the PMU was adopted, so a "config.json" file was used to configure the LibreLane flow, in this case specifying also the die area of the chip (200 µm x 200 µm) to have enough space to place the PMU, the location and orientation of the macro instance, and all the files related to macro cell, e.g. GDS file, LEF file, Verilog header and gate-level netlist files, LIB files, SPEF files.

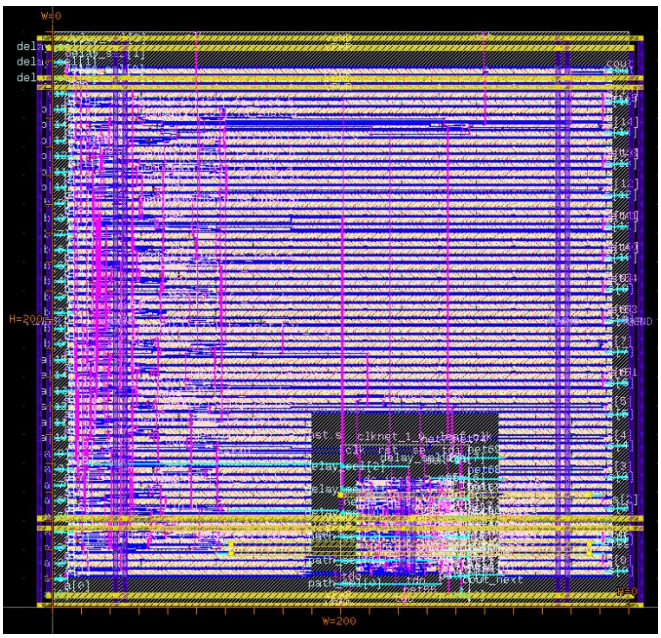

Figure 10. RCA Layout with the PMU Macro Cell inside (H=200 µm, W=200 µm)

After the LibreLane flow finishes, the RCA layout is generated inside the “rca.gds” file and can be visualized with KLayout. A SPICE netlist of the RCA (“rca_pex.spice”) can be obtained using Magic. Finally, a SPICE simulation can be performed using ngspice and another testbench (rca_pex_tb.spice). To allow the SPICE simulation to drive the delay selection signal, delay_sel[2:0] was included in the inputs of the RCA. Also, the Monitor FF output was included in the outputs of the RCA to make it observable to the outside.

Figure 11. shows a plot generated by the testbench with the same signals of **Figure 8.**, but in this case a simple addition which generate a carry-out is performed (A=0xFFFF, B=0x0007) and delay_sel[2:0] = 110 through all the simulation. delay_sel[2:0] = 110 is the first value to make the mismatch between Shadow FF and Capture FF happen, so the timing margin of the critical path is between 11ns and 13ns (using the values reported in in **Figure 9.**).

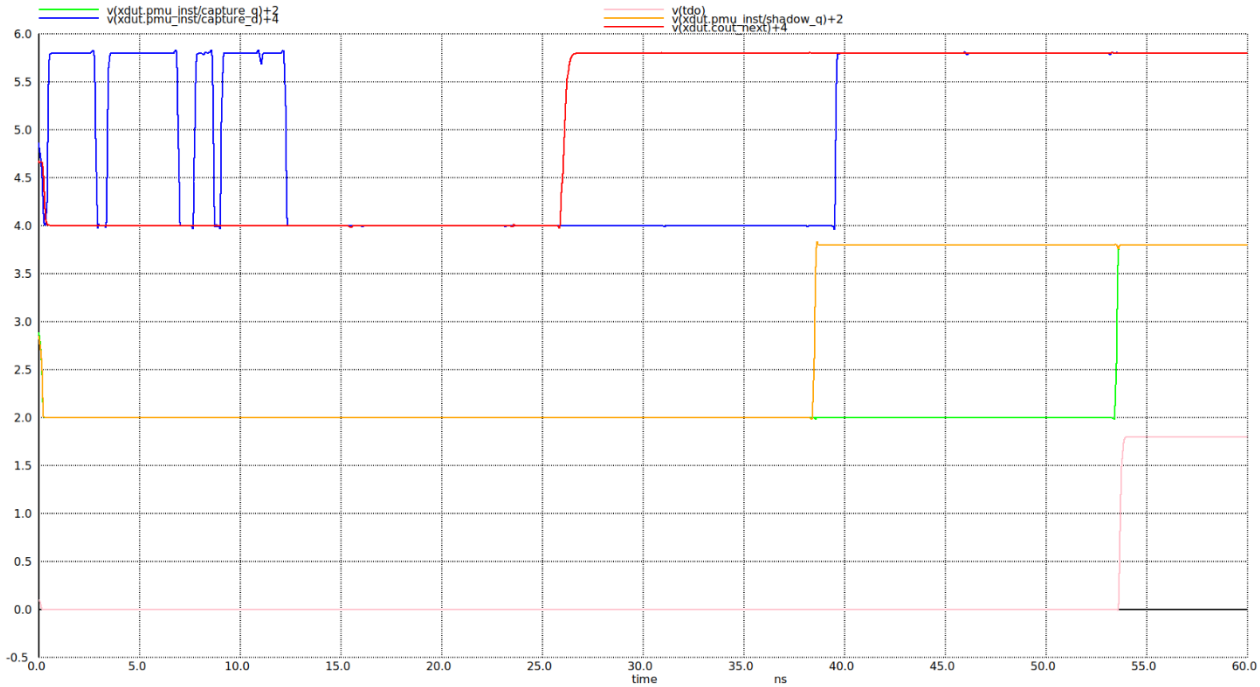

Figure 11. RCA testbench plot

Conclusions

A PMU macro cell was developed and integrated successfully into a simple IC (an RCA) to monitor a critical path (the carry-out). The assignment required to use unfamiliar workflows and software. In particular, the LibreLane documentation was carefully read to generate the two layouts without any errors. Additional Bash scripts were created to automate the adopted flow and to enhance usability and reproducibility. In future works, the number of buffers inside the custom buffers could be changed automatically to increase or decrease the maximum delay a PMU can compute. Also, other and more complex ICs could be used so that more than one PMU could be instantiated inside. The development of a PMC is then the next step as it is necessary to control all the PMUs in an efficient and intended way. As a final step, the tape-out and manufacturing of a PMM (PMU + PMC) in a real hardware could be done.